

Live Coding 02/06 - DFS e BFS

BeeCrows 1076

1. Qual busca ele utiliza?

O enunciado do problema simula uma pessoa desenhando as linhas de um labirinto sem tirar a caneta do papel, precisando voltar pelo mesmo caminho quando atinge um beco sem saída. Esse comportamento de **ir o mais fundo possível em um caminho e depois retroceder (backtracking)** é a definição exata de uma **Busca em Profundidade (DFS - Depth-First Search)**.

Portanto, o algoritmo que descreve o comportamento do problema é a **Busca em Profundidade (DFS)**.

2. A Lógica do Exercício

Se fôssemos implementar a DFS completa apenas para contar os passos, o código funcionaria perfeitamente. Porém, existe uma propriedade dos grafos conexos que nos permite resolver este problema de forma muito mais simples.

Quando você faz uma DFS em um grafo para desenhar caminhos sem repetir linhas que já foram consolidadas, você está essencialmente descobrindo a **Árvore Geradora (Spanning Tree)** daquele grafo.

Uma propriedade fundamental de qualquer árvore com V vértices é que ela possui exatamente $V - 1$ arestas. Para desenhar essa árvore indo e voltando até o ponto inicial (garantindo que passamos por todos os caminhos), nós obrigatoriamente:

1. **Avançamos** por uma aresta nova (1 movimento).
2. **Retrocedemos** por essa mesma aresta ao voltar na pilha de recursão (1 movimento).

Ou seja, **cada aresta única que pertence ao desenho do labirinto será percorrida exatamente 2 vezes** (uma para ir e outra para voltar).

A lógica simplificada: O número total de movimentos decorrentes da DFS será igual a:

$$\text{Total de Movimentos} = \text{Quantidade de Arestas Únicas} \times 2$$

3. Passo a Passo da Solução

Para resolver o exercício seguindo essa lógica simples, o programa faz o seguinte caminho:

Passo 1: Ler a entrada e os casos de teste

O programa lê o número de casos de teste T . Para cada caso, ele recebe o vértice de início (que no fim das contas nem altera o valor do resultado, já que o grafo é totalmente conexo) e os valores de V (vértices) e A (arestas disponíveis na entrada).

Passo 2: Filtrar arestas duplicadas

A entrada do beecrowd pode enviar a mesma aresta mais de uma vez (por exemplo, enviar `1 2` e depois `2 1`, ou repetir `1 2`). Como estamos lidando com um grafo não direcionado, usamos uma matriz de adjacência (`boolean[][] matriz`) para marcar as conexões.

- Ao ler uma aresta entre `u` e `v`, checamos se `matriz[u][v]` já é `true`.
- Se não for, significa que é uma **aresta nova (única)**. Nós a marcamos como `true` (tanto para `u-v` quanto para `v-u`) e incrementamos um contador de arestas únicas.

Passo 3: Calcular o resultado

Após ler todas as arestas do caso de teste, multiplicamos o nosso contador de arestas únicas por 2. Esse resultado é impresso na tela, correspondendo exatamente ao total de segmentos de reta desenhados (ida e volta).

Exemplo Visual do Processo

Imagine um grafo em formato de triângulo com os vértices 0, 1 e 2:

- Você entra no labirinto pelo `0`.

- Vai de 0 para 1 (Aresta 1 - Ida).
- De 1, vai para 2 (Aresta 2 - Ida).
- De 2, tenta ir para 0, mas 0 já foi visitado. Você pega o caminho de volta de 2 para 1 (Aresta 2 - Volta).
- De 1, volta para 0 (Aresta 1 - Volta).

Foram 2 arestas exclusivas usadas no caminho, gerando $2 \times 2 = 4$ movimentos no total.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputS
treamReader(System.in));
        String line = reader.readLine();
        if (line == null) return;

        int T = Integer.parseInt(line.trim());

        while (T > 0) {
            T--;
            int inicio = Integer.parseInt(reader.readLine().t
rim());

            // Divide a linha usando qualquer quantidade de e
spaços em branco como separador
            String[] partesVA = reader.readLine().trim().spli
t("\\s+");

            int V = Integer.parseInt(partesVA[0]);
            int A = Integer.parseInt(partesVA[1]);
        }
    }
}
```

```

        boolean[][] matrizAdjacencia = new boolean[V][V];
        int arestasUnicas = 0;

        for (int i = 0; i < A; i++) {
            String[] partesAresta = reader.readLine().trim().split("\\s+");
            int u = Integer.parseInt(partesAresta[0]);
            int v = Integer.parseInt(partesAresta[1]);

            if (!matrizAdjacencia[u][v]) {
                matrizAdjacencia[u][v] = true;
                matrizAdjacencia[v][u] = true;
                arestasUnicas++;
            }
        }
        System.out.println(arestasUnicas * 2);
    }
}

```

BeeCrowd 1081

1. A Lógica de Formatação

O desafio aqui não é o algoritmo de busca em si, mas a maneira de estruturar a saída:

- Cada nível de profundidade na recursão adiciona **2 espaços em branco** antes de imprimir a aresta.
- Se a aresta leva a um vértice que **ainda não foi visitado**, ela faz parte do caminho de descida. Imprimimos `u-v pathR(G,v)` e descemos na recursão (aumentando o recuo).

- Se a aresta leva a um vértice que **já foi visitado**, ela é apenas uma aresta de retorno ou cruzamento. Imprimimos apenas `u-v` (com o recuo atual) e **não** descemos na recursão.
- Cada componente do grafo que gerar alguma linha na tela deve ser finalizado com uma **linha em branco** extra.

2. Passo a Passo do Código

Vamos quebrar a execução do programa em etapas bem definidas:

Passo 1: Leitura e Inicialização

O programa recebe o número de casos de teste. Para cada caso, ele lê V (vértices) e E (arestas).

Construímos uma matriz de adjacência quadrada de tamanho $V \times V$ para representar o grafo direcionado e um array de booleanos `visitado` de tamanho V .

Passo 2: O Loop Principal (Varrer a Floresta)

Como o grafo pode ter pedaços isolados, precisamos garantir que todos os vértices sejam testados como possíveis pontos de partida. Fazemos isso com um laço de `0` até `V-1`:

```
for (int i = 0; i < V; i++) {
    if (!visitado[i]) {
        temSaida = false;
        dfs(i, 0); // Começa a busca com 0 espaços de recuo
        if (temSaida) {
            System.out.println(); // Linha em branco após fec
har o componente
        }
    }
}
```

A variável global `temSaida` serve para monitorar se aquele vértice inicial realmente tinha alguma aresta saindo dele. Se ele estivesse completamente isolado, não imprimimos nada (nem a linha em branco).

Passo 3: A Função Recursiva `dfs(u, espaços)`

Quando entramos na função para o vértice `u`, nós o marcamos imediatamente como `visitado = true`.

A partir daí, olhamos para todos os possíveis vizinhos `v` (de `0` a `V-1`) em ordem crescente:

1. **Existe conexão?** Se `grafo[u][v]` for `true`, ativamos `temSaida = true`.
2. **O vizinho `v` não foi visitado?** * Montamos o recuo de espaços atual.
 - Imprimimos: `u-v pathR(G,v)`
 - Chamamos recursivamente a função para o vizinho, **aumentando o recuo**: `dfs(v, espaços + 2)`.
3. **O vizinho `v` já tinha sido visitado antes?**
 - Montamos o recuo de espaços atual.
 - Imprimimos apenas: `u-v`
 - Não fazemos chamada recursiva; o laço simplesmente passa para o próximo vizinho.

Exemplo Prático de Execução

Imagine um minigrafo direcionado com 3 vértices: `0 -> 1` e `1 -> 2`, começando pelo vértice `0`.

1. `dfs(0, 0)` é chamada. `visitado[0] = true`.
 - Verifica vizinho `1`: não está visitado.
 - Imprime (com 0 espaços): `0-1 pathR(G,1)`
 - Chama `dfs(1, 2)`.
2. `dfs(1, 2)` em execução. `visitado[1] = true`.
 - Verifica vizinho `2`: não está visitado.
 - Imprime (com 2 espaços): `1-2 pathR(G,2)`
 - Chama `dfs(2, 4)`.
3. `dfs(2, 4)` em execução. `visitado[2] = true`.

- Não tem vizinhos saindo dele. A função encerra e volta para o nível anterior.
4. O laço de `dfs(1, 2)` termina. A função encerra e volta para o nível anterior.
 5. O laço de `dfs(0, 0)` termina. `temSaida` é `true`, então quebra uma linha vazia.

```
package DFS-BeeCrowd1081;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Main {
    private static int V;
    private static boolean[][] grafo;
    private static boolean[] visitado;
    private static boolean temSaida;
    private static StringBuilder sb;

    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        String line = reader.readLine();
        if (line == null) return;

        int N = Integer.parseInt(line.trim());

        for (int caso = 1; caso <= N; caso++) {
            String[] partesVE = reader.readLine().trim().split("\\s+");
            V = Integer.parseInt(partesVE[0]);
            int E = Integer.parseInt(partesVE[1]);

            grafo = new boolean[V][V];
            visitado = new boolean[V];
```

```

        for (int i = 0; i < E; i++) {
            String[] partesAresta = reader.readLine().trim().split("\\s+");
            int u = Integer.parseInt(partesAresta[0]);
            int v = Integer.parseInt(partesAresta[1]);
            grafo[u][v] = true;
        }

        // Imprime o cabeçalho do caso
        System.out.println("Caso " + caso + ":");

        for (int i = 0; i < V; i++) {
            if (!visitado[i]) {
                temSaida = false;
                sb = new StringBuilder();

                // Executa a busca a partir da raiz do componente (começa com 2 espaços)
                dfs(i, 2);

                // Se este componente gerou alguma aresta impressa
                if (temSaida) {
                    System.out.print(sb.toString());
                    System.out.println(); // Força a linha a em branco IMEDIATAMENTE após o segmento
                }
            }
        }
    }

    private static void dfs(int u, int espaços) {
        visitado[u] = true;

```



```

// Gera a string de recuo de forma simples
StringBuilder recuo = new StringBuilder();
for (int i = 0; i < espaços; i++) {
    recuo.append(" ");
}

for (int v = 0; v < V; v++) {
    if (grafo[u][v]) {
        temSaida = true;
        if (!visitado[v]) {
            sb.append(recuo).append(u).append("-").append(v).append(" pathR(G, ").append(v).append(")\n");
            dfs(v, espaços + 2);
        } else {
            sb.append(recuo).append(u).append("-").append(v).append("\n");
        }
    }
}
}
}
}

```

BeeCrowd 2429

1. Entendendo o Enunciado e a Lógica

O problema nos dá um país com N cidades e exatamente **N rodovias de mão única (direcionadas)**. Cada rodovia liga duas cidades diretamente. O objetivo é determinar se, a partir de **qualquer** cidade, é possível alcançar **todas** as outras cidades usando essas rodovias.

Na teoria dos grafos, um grafo direcionado onde todos os vértices conseguem alcançar todos os outros é chamado de **Grafo Fortemente Conexo**.

Propriedade dos Graus

Como o número de arestas (rodovias) é rigorosamente **igual** ao número de vértices (cidades) e o grafo precisa ser fortemente conexo, existe uma restrição matemática muito forte sobre os graus de entrada e saída dos vértices:

1. Para que todo mundo consiga ir e vir, nenhuma cidade pode ser um "beco sem saída" (grau de saída = 0) e nenhuma cidade pode ser inacessível (grau de entrada = 0).
2. Como temos exatamente N arestas para distribuir entre N cidades, a única configuração possível para o grafo ser fortemente conexo é que **cada cidade possua exatamente 1 rodovia saindo dela e exatamente 1 rodovia chegando nela**.

Se qualquer cidade tiver 2 rodovias saindo e outra tiver 0 saindo, o ciclo se quebra e o grafo deixa de ser fortemente conexo. Portanto, o problema pode ser resolvido simplesmente checando se o **grau de entrada** (ou de saída) de todas as cidades é igual a 1.

No entanto, para consolidar o aprendizado em algoritmos de percurso, podemos resolvê-lo utilizando uma **Busca em Largura (BFS)** clássica. Partindo de um nó qualquer (por exemplo, a cidade 1), disparamos a BFS. Se ela conseguir visitar todas as N cidades e o número total de rodovias fornecidas for consistente com o fluxo cíclico, o grafo é fortemente conexo e a resposta é **S**. Caso contrário, a resposta é **N**.

2. Passo a Passo do Algoritmo com BFS

1. **Leitura:** Lemos o valor de N. Em seguida, criamos um vetor (ou lista de listas) para representar o grafo direcionado e um vetor para contar o grau de entrada de cada cidade.
2. **Construção do Grafo:** Populamos nossa lista de adjacências com as rodovias direcionadas de $U \rightarrow V$.
3. **Busca em Largura (BFS):** Disparamos uma BFS a partir da cidade 1. Usamos uma fila (**Queue**) e um vetor de **visitado**.
4. **Verificação:** Contamos quantas cidades a BFS conseguiu alcançar. Se o contador for igual a N e todas as cidades possuírem grau de entrada igual a 1, significa que o circuito fecha perfeitamente. Imprimimos **S**, senão **N**.

```

package BFS-BeeCrowd2429;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;

public class Main {
    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputS
treamReader(System.in));
        String line = reader.readLine();
        if (line == null) return;

        int N = Integer.parseInt(line.trim());

        // Inicializa a lista de adjacência para o grafo dire
cionado
        ArrayList<ArrayList<Integer>> grafo = new ArrayList<>
();
        for (int i = 0; i <= N; i++) {
            grafo.add(new ArrayList<>());
        }

        // Vetor para monitorar o grau de entrada de cada cid
ade
        int[] grauEntrada = new int[N + 1];

        // Leitura das N rodovias
        for (int i = 0; i < N; i++) {
            String[] partes = reader.readLine().trim().split
("\\s+");

```

```

        int u = Integer.parseInt(partes[0]);
        int v = Integer.parseInt(partes[1]);

        grafo.get(u).add(v); // Rodovia de mão única: U -
> V
        grauEntrada[v]++;    // V recebe uma rodovia vind
a de U
    }

    // Primeiramente, valida se existe alguma cidade sem
rodovia chegando
    boolean estruturaValida = true;
    for (int i = 1; i <= N; i++) {
        if (grauEntrada[i] != 1) {
            estruturaValida = false;
            break;
        }
    }

    // Se a contagem de graus falhou, nem precisamos roda
r a busca
    if (!estruturaValida) {
        System.out.println("N");
        return;
    }

    // --- Execução da Busca em Largura (BFS) ---
    boolean[] visitado = new boolean[N + 1];
    Queue<Integer> fila = new LinkedList<>();

    // Começa a busca a partir da cidade 1
    fila.add(1);
    visitado[1] = true;
    int cidadesVisitadas = 1;

    while (!fila.isEmpty()) {

```

```

        int atual = fila.poll();

        // Varre os vizinhos direcionados da cidade atual
        for (int vizinho : grafo.get(atual)) {
            if (!visitado[vizinho]) {
                visitado[vizinho] = true;
                cidadesVisitadas++;
                fila.add(vizinho);
            }
        }
    }

    // Se a BFS conseguiu tocar todas as N cidades, o grafo é fortemente conexo
    if (cidadesVisitadas == N) {
        System.out.println("S");
    } else {
        System.out.println("N");
    }
}
}

```

Beecrowd 2440

1. Entendendo o Enunciado e a Lógica

O problema nos diz que temos N pessoas e M relações de parentesco. Cada relação diz que a pessoa U é parente da pessoa V. O parentesco é transitivo: se A é parente de B e B é parente de C, então A é parente de C.

O objetivo é descobrir **quantas famílias independentes existem** na população.

Como modelar isso como um Grafo?

- Cada pessoa é um **vértice** (nó).

- Cada relação de parentesco é uma **aresta não direcionada** (se U é parente de V, V também é parente de U).

O que o problema quer, na verdade, é que você conte quantos **componentes conexos** (grupos isolados) existem no grafo.

2. Como a BFS resolve isso sozinha?

A estratégia usando a Busca em Largura é muito elegante e direta:

1. Nós começamos com um contador de famílias igual a `0` e um vetor de `visitado` para todas as pessoas.
2. Fazemos um laço de `1` até `N` para testar cada pessoa.
3. Se encontrarmos uma pessoa que **ainda não foi visitada**:
 - Significa que descobrimos uma **nova família**! Incrementamos o contador de famílias em +1.
 - Disparamos uma **BFS** a partir dessa pessoa.
 - A BFS vai entrar na fila, olhar todos os parentes diretos dela, depois os parentes dos parentes, e assim por diante. Cada pessoa que a BFS tocar é marcada como `visitado = true`.
 - Quando a fila da BFS esvaziar, significa que toda aquela árvore genealógica foi mapeada e marcada.
4. O laço principal continua. Se a próxima pessoa já estiver marcada como visitada, o código simplesmente a ignora, pois ela já faz parte de uma família que computamos antes.

Ao final do laço, o nosso contador terá o número exato de famílias.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
```

```

public class Main {
    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputS
treamReader(System.in));
        String line = reader.readLine();
        if (line == null) return;

        String[] partesNM = line.trim().split("\\s+");
        int N = Integer.parseInt(partesNM[0]); // Número de p
essoas
        int M = Integer.parseInt(partesNM[1]); // Número de r
elações

        // Inicializa a lista de adjacência para o grafo não
direcionado
        ArrayList<ArrayList<Integer>> grafo = new ArrayList<>
();
        for (int i = 0; i <= N; i++) {
            grafo.add(new ArrayList<>());
        }

        // Leitura das M relações de parentesco
        for (int i = 0; i < M; i++) {
            String[] partesAresta = reader.readLine().trim().
split("\\s+");
            int u = Integer.parseInt(partesAresta[0]);
            int v = Integer.parseInt(partesAresta[1]);

            // Grafo não direcionado (relação mútua)
            grafo.get(u).add(v);
            grafo.get(v).add(u);
        }

        boolean[] visitado = new boolean[N + 1];
        int totalFamilias = 0;
    }
}

```

```

        // Laço principal para varrer todas as pessoas de 1 a
N
        for (int i = 1; i <= N; i++) {
            // Se encontramos alguém que não foi visitado, ac
hamos uma nova família!
            if (!visitado[i]) {
                totalFamilias++;

                // --- Execução da BFS para marcar toda a fam
ília de 'i' ---
                Queue<Integer> fila = new LinkedList<>();
                fila.add(i);
                visitado[i] = true;

                while (!fila.isEmpty()) {
                    int atual = fila.poll();

                    // Varre todos os parentes diretos da pes
soa atual
                    for (int parente : grafo.get(atual)) {
                        if (!visitado[parente]) {
                            visitado[parente] = true;
                            fila.add(parente); // Adiciona na
fila para buscar os parentes dele
                        }
                    }
                }
                // --- Fim da BFS para esta família ---
            }
        }

        // Imprime a quantidade de famílias independentes enc
ontradas
        System.out.println(totalFamilias);

```



```
}  
}
```